



Building a Google Chat Bot with Express and TypeScript

This guide shows how to create a Google Chat bot (HTTP-based) step-by-step using Node.js/Express and TypeScript. It covers project setup, authentication, event handling, calling an AI endpoint, building Chat cards, sending messages, and deployment. We assume the bot will be installed in select Google Workspace domains (via the Marketplace) and will need access to workspace resources like Drive or Calendar. Each step includes code snippets (TS/JSON) and references to the official Google documentation.

1. Set Up Google Cloud Project and APIs

- **Create a new Google Cloud project.** In the [Cloud Console](#), click *Select a project* (top bar) and create a project.
- **Enable needed APIs:** Go to **APIs & Services > Library** and enable:
 - **Google Chat API** ¹.
 - **Google Workspace Marketplace SDK** ² (for controlling installation/visibility).
 - Any other Workspace APIs your bot will use (e.g. Google Drive API, Calendar API).
- **Configure OAuth consent screen:** Under **APIs & Services > OAuth consent screen**, set *User Type* (typically *Internal* for a Workspace-only app) and fill in app info. This screen will show when users (or admin) authorize scopes your app needs ³.
- **Create credentials:**
 - **Service account:** In **IAM & Admin > Service Accounts**, create a service account for your bot ⁴. Download its JSON key (click *Keys > Add Key > JSON*) and save it securely (e.g. `service-account.json`) ⁵. The bot uses this to authenticate to Google APIs.
 - **OAuth 2.0 client ID:** If your bot will access **per-user data** (Drive, Calendar, etc.) via user consent, create an OAuth client ID (type *Web application*) under **APIs & Services > Credentials**. Set your app's redirect URI(s) (e.g. `https://your-app/callback`). Use the generated client ID/secret in your code for OAuth flows.

(Support: "Before using Google APIs, you need to turn them on in a Google Cloud project" ². Each Chat app needs its own project with the Chat API enabled ³.)

2. Configure Google Workspace Marketplace SDK

- **Enable the Marketplace SDK:** In **APIs & Services > Library**, search for *Google Workspace Marketplace SDK* and enable it ².
- **Marketplace SDK configuration:** In **APIs & Services > Google Workspace Marketplace SDK**, set up your app's listing:
 - **App visibility:** Choose *Private* (domain-only) if only your organization should see it, or *Unlisted/Public* if wider. For selected companies, you'd likely use a Private or Unlisted setting ⁶ ⁷.
 - **Installation settings:** Decide if individual users or only admins can install. For Chat apps, the default is *Individual + Admin* (users can install directly from Chat) ⁷.

- **Supported Google Workspace apps:** Ensure *Google Chat* is checked.
- **OAuth scopes:** List the scopes your bot needs (e.g. `auth/chat.bot` for bot actions, plus Drive/Calendar scopes if used).
- **Developer info:** Fill in contact email, logo, etc.

Administrators in target organizations may need to **whitelist** your app if their domain uses allowlists ⁸. You can specify tester emails/groups in the *Visibility* section of the Chat API config (next step) so only they can install it initially ⁹.

(Admins can restrict which apps can be installed; use allowlists if necessary ⁸. Use the Marketplace SDK to manage visibility and installation options ⁶ ⁷.)

3. Create a Chat App in Cloud Console (Chat API configuration)

- **Go to the Chat API configuration:** In the Cloud Console, navigate to **APIs & Services > Google Chat API**, then click **Configuration** ¹⁰.
- **Application info:** Enter your bot's **App name**, **Avatar URL** (HTTPS image URL, e.g. 256×256 PNG), and short **Description** ¹¹.
- **Interactive features:** Toggle *Enable interactive features* on, then under **Functionality** check:
 - *Receive 1:1 messages* (allows direct messages with the bot) ¹².
 - *Join spaces and group conversations* (lets users @mention or add the bot in group chats) ¹².
- **Connection settings:** For *Authentication*, select **HTTP endpoint URL** and enter your bot's webhook URL (e.g. `https://your-app-url/chatbot`) ¹³. This is where Chat will POST events. (Do **not** use Apps Script unless you're building an Apps Script add-on.)
- **Triggers:** By default, Chat apps handle certain events. You can optionally add triggers under *Connection settings > Triggers*:
 - *Added to space* (`onAddToSpace`),
 - *Message* (`onMessage`),
 - *Card clicked* (`onCardClick`), etc ¹⁴.
- **Slash commands (optional):** Under *Interactive features*, add **Slash commands** and **Quick commands** if you want users to trigger your bot via slash menus ⁹. For each command, give a name (like `/status`) and description.
- **Visibility (test users):** In *Interactive features > Visibility*, add up to 5 users or groups (e.g. your email) who can install and test the bot before it's published ⁹.
- **Save your settings.** Once saved, the Chat app configuration is active. Users you listed can now install your bot in Chat by @mentioning it or via the *Chat apps* menu.

(This matches the official setup: "In Cloud console, go to the Chat API page and ... configure the Chat app: name, avatar, description, enable features" ¹⁰. The UI also lets you define slash commands and event callbacks ⁹.)

4. Authentication

4.1 Service Account (Bot Identity)

Your bot needs to authenticate to the Chat API when *calling* Google (e.g. for proactive messages). Use the service account and JSON key you created:

- **Load credentials:** In your code, load the service account key. You can use the [google-auth-library](#) or [@google-cloud](#) libraries. For example, with `googleapis`:

```
import {GoogleAuth} from 'google-auth-library';
const auth = new GoogleAuth({
  credentials: require('./service-account.json'),
  scopes: ['https://www.googleapis.com/auth/chat.bot']
});
const chatClient = google.chat({version: 'v1', auth});
```

- **Scopes:** Use the `chat.bot` scope for bot API calls. This scope doesn't require admin approval ¹⁵. (Scopes starting with `chat.app.` do require a one-time admin approval ¹⁵, but for most bots `chat.bot` is enough.)

4.2 Domain-Wide Delegation (Optional)

If your bot needs to **access user data** (e.g. a user's Drive files or Calendar) without prompting each user, enable Domain-Wide Delegation (DWD):

- In **IAM & Admin > Service Accounts**, edit the service account and check *Enable G Suite Domain-wide Delegation*. Copy the client ID.
- In your Google Workspace Admin console, go to *Security > API Controls > Domain-wide Delegation* and **Add new**. Enter the client ID and the scopes your bot needs (e.g. `https://www.googleapis.com/auth/drive`).
- With DWD, the service account can impersonate users. In code, you would use something like:

```
const authClient = new GoogleAuth({
  credentials: require('./service-account.json'),
  scopes: ['https://www.googleapis.com/auth/drive'],
  clientOptions: { subject: 'some-user@yourdomain.com' } // impersonate
  this user
});
```

This gives you a token acting as that user.

(Docs: “If you're a domain admin, you can grant domain-wide delegation ... to authorize a Chat app's service account to access your users' data without requiring each user to give consent. The service account can impersonate a user account” ¹⁶.)

4.3 OAuth 2.0 User Consent (Optional)

If you choose not to use DWD, you can do a standard OAuth 2.0 flow to get each user's consent:

1. Direct users to the Google OAuth 2.0 consent URL with your client ID and required scopes.
2. Handle the redirect to capture the authorization code.
3. Exchange the code for tokens (using `googleapis` or similar).
4. Store the `access_token` (and refresh token) in your database for that user.
5. Use the user's token in API calls when needed.

This is standard Google OAuth for web apps (see [Google OAuth2 docs](#)).

(If your bot only uses service-account authorization (app-level, e.g. chat.bot), you don't need this user OAuth part.)

4.4 Verify Incoming Requests

Important: When Google Chat calls your webhook, it includes a bearer token you must verify. In each request to your Express endpoint, Google sets:

```
Authorization: Bearer <JWT>
User-Agent: Google-Dynamite
```

This JWT is an **OIDC ID token** signed by Google. Verify it as follows:

- In Node, use `google-auth-library`'s `OAuth2Client.verifyIdToken()`. For example:

```
import {OAuth2Client} from 'google-auth-library';
const client = new OAuth2Client();

async function verifyGoogleChatToken(idToken: string, audience: string) {
  const ticket = await client.verifyIdToken({ idToken, audience });
  const payload = ticket.getPayload();
  // Verify issuer and audience
  return payload?.email_verified
    && payload.email === 'chat@system.gserviceaccount.com'
    && payload.aud === audience;
}
```

Here, **audience** should match the URL of your endpoint (the *Authentication audience* you set in Chat config).

(Google example: They verify that `email_verified` is true and `email` equals `chat@system.gserviceaccount.com` ¹⁷ ¹⁸.)

If verification fails, your app should reject with 401. (If you deploy to Cloud Run/Functions, you can also use IAM auth.)

(Ref: “Google Chat includes a bearer token in the Authorization header of every request to your endpoint... verify it using Google’s client libraries” ¹⁹ ¹⁷ .)

5. Webhook Endpoint with Express + TypeScript

Set up an Express server to receive Chat events. For example:

```
import express from 'express';
import bodyParser from 'body-parser';
import {OAuth2Client} from 'google-auth-library';

const app = express();
app.use(bodyParser.json());

// Middleware to verify Google Chat JWT
const CLIENT = new OAuth2Client();
async function verifyToken(req, res, next) {
  try {
    const authHeader = (req.get('Authorization') || '');
    const token = authHeader.replace('Bearer ', '');
    const ticket = await CLIENT.verifyIdToken({
      idToken: token,
      audience: 'https://your-ngrok-or-cloud-run-url',
    });
    const payload = ticket.getPayload();
    if (payload?.email_verified && payload.email ===
'chat@system.gserviceaccount.com') {
      return next();
    } else {
      throw new Error('Invalid Chat token');
    }
  } catch (err) {
    console.error('Chat authentication failed:', err);
    res.sendStatus(401);
  }
}

app.post('/chatbot', verifyToken, async (req, res) => {
  const event = req.body; // Chat event JSON
  // TODO: Handle the event (see next section)
});
```

```
const PORT = process.env.PORT || 8080;
app.listen(PORT, () => console.log(`Server listening on port ${PORT}`));
```

- This code sets up `POST /chatbot` as the webhook (use the same path you configured in Cloud Console).
- It verifies each request's JWT using `verifyIdToken()` as discussed ¹⁷.
- On success, it calls the next handler; on failure, it sends 401.
- The endpoint will receive JSON events from Chat (with body fields like `type`, `message`, etc.).

Deploy this Express app to a secure HTTPS environment (Cloud Run, App Engine, GKE, etc.). For **local testing**, run `ngrok http 8080` and use the provided `https://*.ngrok.io/chatbot` as the Chat API endpoint.

6. Handling Events (`MESSAGE`, `ADDED_TO_SPACE`, `CARD_CLICKED`, etc.)

When a user interacts with your bot, Chat sends an *event* object. Key fields include:

- `event.type`: type of event (string). Common values:
 - `"MESSAGE"` – user sent a message or slash command to the bot.
 - `"ADDED_TO_SPACE"` – bot was added to a space.
 - `"REMOVED_FROM_SPACE"` – bot was removed (rarely needed).
 - `"CARD_CLICKED"` – user clicked a card widget or submitted a form.
- `event.message`: an object containing message details (when `type=="MESSAGE"`). Includes `text` (user's message text), `sender` (user info), etc.
- `event.space`: info about the space (chat room) where this happened.

Your handler should check `event.type` and act accordingly. For example:

```
app.post('/chatbot', verifyToken, async (req, res) => {
  const event = req.body;
  switch (event.type) {
    case 'ADDED_TO_SPACE':
      // e.g. send a welcome card/message
      await handleAdded(event, res);
      break;
    case 'MESSAGE':
      await handleMessage(event, res);
      break;
    case 'CARD_CLICKED':
      await handleCardClick(event, res);
      break;
    // handle other event types if needed
  }
});
```

```

    default:
      // Unrecognized event
      res.sendStatus(400);
    }
  });

```

Example behaviors:

- On `ADDED_TO_SPACE`, you usually send a welcome message or card. (Users added your bot; see [49] example.)
- On `MESSAGE`, parse the user's text (`event.message.text`). If it's a slash command (like `/help`), respond accordingly. Often you'll call your AI endpoint here.
- On `CARD_CLICKED`, process any inputs from the card form or button that was clicked. Chat delivers form values in the event payload. Respond with another card or message as needed.

(Reference: Google's docs note that `ADDED_TO_SPACE` and `MESSAGE` are common events and suggest sending a welcome or reply respectively ²⁰ ²¹ .)

7. Processing Messages and Calling the AI Endpoint

In the `'MESSAGE'` case, extract the user's message text:

```

const userText = event.message.text; // The raw string, e.g. "/ask What is the
weather?"

```

Send this text to your prebuilt AI endpoint (a hypothetical REST API) and get the response. For example, using `axios`:

```

import axios from 'axios';

async function queryAI(userMessage: string) {
  const resp = await axios.post('https://your-ai-endpoint.com/query', {
    text: userMessage
  });
  return resp.data; // Could be string or JSON
}

async function handleMessage(event, res) {
  const aiResponse = await queryAI(event.message.text);
  if (typeof aiResponse === 'string') {
    // Plain text response: send simple message
    res.json({ text: aiResponse });
  } else {
    // Assume JSON with cards or structured data, send as cards
    // e.g. if aiResponse = { cardsV2: [ ... ] }

```

```

    res.json(aiResponse);
  }
}

```

- If the AI returns plain text, you can simply reply with `{ text: "..."}` .
- If it returns structured data (e.g. an object with `cardsV2`), you can send that JSON directly as the reply.

Always respond promptly. You can use `actionResponse: { type: "NEW_MESSAGE" }` to indicate a normal reply. (By default, returning a JSON object with `text` or `cardsV2` creates a new message.)

(This step is app-specific; just ensure you parse both string and JSON responses. There's no direct Google doc for this AI part, but treat your AI like any web service.)

8. Building Rich UIs with Google Chat Cards

Google Chat **Cards** let you create interactive, rich messages. A card is a JSON object with a header, sections, and widgets (text, images, buttons, inputs, etc.) ²². Here's an example of a simple card reply with text and a button:

```

{
  "actionResponse": { "type": "NEW_MESSAGE" },
  "cardsV2": [{
    "cardId": "helloCard",
    "card": {
      "header": {
        "title": "Welcome to the Bot",
        "subtitle": "Choose an option below"
      },
      "sections": [
        {
          "widgets": [
            {
              "textParagraph": {
                "text": "This is a **rich card** message with a button."
              }
            },
            {
              "buttons": [
                {
                  "textButton": {
                    "text": "Click me",
                    "onClick": {
                      "action": {
                        "function": "button_click",
                        "parameters": [

```


[illegible]

- **Sections & Widgets:** Cards have one or more *sections*. Each section contains *widgets* like `textParagraph`, `image`, `buttons`, `TextInput`, etc.
- **Buttons/Actions:** The `onClick.action.function` specifies a function name that's sent back in the next event (as `CARD_CLICKED`). You handle it in your code. You can also use `openLink` to open URLs.
- **Text Inputs:** To collect user input, use `TextInput` or `SelectionInput` widgets inside a `Form`. When the user submits (typically via an action button), Chat sends a `CARD_CLICKED` event with the input data.

Send card replies by returning JSON with `"cardsV2": [ ... ]` as above. Google provides a [Card Builder](#) tool to visually design cards.

(For details on card components, see Google's [Build cards for Chat apps](#) guide ²².)

9. Sending Proactive Messages (REST API)

Your bot can *initiate* messages (not just reply) by calling the Chat REST API. For example, use the Node client library:

```
import {google} from 'googleapis';
const chat = google.chat('v1');

// Assume `auth` is a GoogleAuth instance authorized with service account
// (as set up in section 4.1)
async function sendMessage(spaceName: string, text: string) {
  const res = await chat.spaces.messages.create({
    parent: spaceName,    // e.g. 'spaces/AAAA...'
    requestBody: { text }
  });
  console.log('Message sent:', res.data);
}
```

```
// Usage:
await sendMessage('spaces/SPACE_ID', 'This is a proactive alert!');
```

Replace `'spaces/SPACE_ID'` with the actual space name or ID. You can find space IDs via the Chat API or from URLs.

Google's example:

```
const result = await chatClient.spaces.messages.create({
  parent: 'spaces/SPACE_NAME',
  requestBody: { text: 'Hello, world!' }
});
```

23 24 .

This is useful for scheduled notifications or updates. Ensure your bot is a member of the space (or create a new space with `spaces.create()` if needed).

(Ref: Google Chat API `spaces.messages.create` method as shown above 23 24 .)

10. Testing and Deployment

- **Local testing:** Use a tool like [ngrok](#) to expose your local server. For example, run `ngrok http 8080`, and set the Chat endpoint to the `https://*.ngrok.io/chatbot` URL. This lets Chat deliver events to your dev machine.
- **Verify events:** Use the Google Chat API's *Test and Debug* tab (in Cloud Console) to send sample events. Also manually send messages from Chat to the bot. Always check your server logs for incoming payloads.
- **JWT Verification:** As mentioned, verify every incoming JWT. If using Cloud Run/Functions, you can configure IAM to only allow the `chat@system.gserviceaccount.com` invoker, but manual JWT check is safer.
- **Deployment:** Deploy your Express app to production:
- **Cloud Run:** Containerize or use Cloud Buildpack, point the service to listen on `$PORT` (8080). HTTPS is automatic.
- **App Engine:** Use the Node.js App Engine standard/flex environment.
- **Other:** Any HTTPS server (ensure SSL).
- **Update Chat endpoint:** After deployment, update the **Connection settings** URL in the Chat API console to point to your production URL.
- **Logging:** In Chat API config, enable *Log errors to Logging* for debugging 25 . Monitor error logs (Stackdriver/Cloud Logging).

(The Chat API docs recommend testing interactive features after saving the configuration 26 .)

11. Best Practices

- **Performance:** Keep responses quick. If your AI call might be slow, consider replying with an interim “working...” message (using `actionResponse: {type: "ACKNOWLEDGE"}`) before returning the final answer.
- **Error Handling:** Always `catch` errors in your handlers. On failure, send an error card or message. Return HTTP 500 if your service failed unexpectedly (so Chat knows it didn't get a valid response).
- **Security:** Don't expose secrets. Use environment variables or Secret Manager for credentials. Restrict service account scopes to only what's needed.
- **Auth Tokens:** Cache OAuth tokens and refresh them as needed. For service accounts, you can reuse JWT client.
- **Domain Restriction:** Since only selected companies install this bot, publish it as a *private* app or unlisted. Administrators may need to add it to allowlists ⁸.
- **Cleanup on Removal:** If you handle `REMOVED_FROM_SPACE`, use it to delete any associated data or cancel subscriptions to avoid leaks.
- **User Experience:** Provide help (`/help`) and clear instructions. For cards, use concise text and Alt text on images for accessibility.

By following these steps and examples, you can build a fully featured Google Chat bot in TypeScript that interacts via messages, cards, and external AI services. For more details, refer to Google's documentation: configuring the Chat API ¹⁰, authentication guides ¹⁵ ¹⁹, and card/message reference ²² ²³.

Sources: Google Chat developer guides and API references ¹⁰ ¹⁵ ¹⁹ ²¹ ²³ (see specific lines cited above). These describe Chat app setup, authentication, event types, message formats, and more.

¹ Node.js quickstart | Google Chat | Google for Developers

<https://developers.google.com/workspace/chat/api/guides/quickstart/nodejs>

² ⁶ ⁷ ⁸ Configure your app in the Google Workspace Marketplace SDK | Google for Developers

<https://developers.google.com/workspace/marketplace/enable-configure-sdk>

³ Configure the Google Chat API | Google for Developers

<https://developers.google.com/workspace/chat/configure-chat-api>

⁴ ⁵ ¹⁵ ²³ ²⁴ Authenticate as a Google Chat app | Google for Developers

<https://developers.google.com/workspace/chat/authenticate-authorize-chat-app>

⁹ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ²⁵ ²⁶ Configure a Google Chat app | Google Workspace add-ons | Google for Developers

<https://developers.google.com/workspace/add-ons/chat/configure>

¹⁶ Authenticate and authorize as a Google Chat user | Google for Developers

<https://developers.google.com/workspace/chat/authenticate-authorize-chat-user>

¹⁷ ¹⁸ ¹⁹ Verify requests from Google Chat | Google for Developers

<https://developers.google.com/workspace/chat/verify-requests-from-chat>

²⁰ ²¹ Receive and respond to user interactions | Google Chat | Google for Developers

<https://developers.google.com/workspace/chat/receive-respond-interactions>

22 Build cards for Google Chat apps | Google for Developers

<https://developers.google.com/workspace/chat/design-components-card-dialog>